

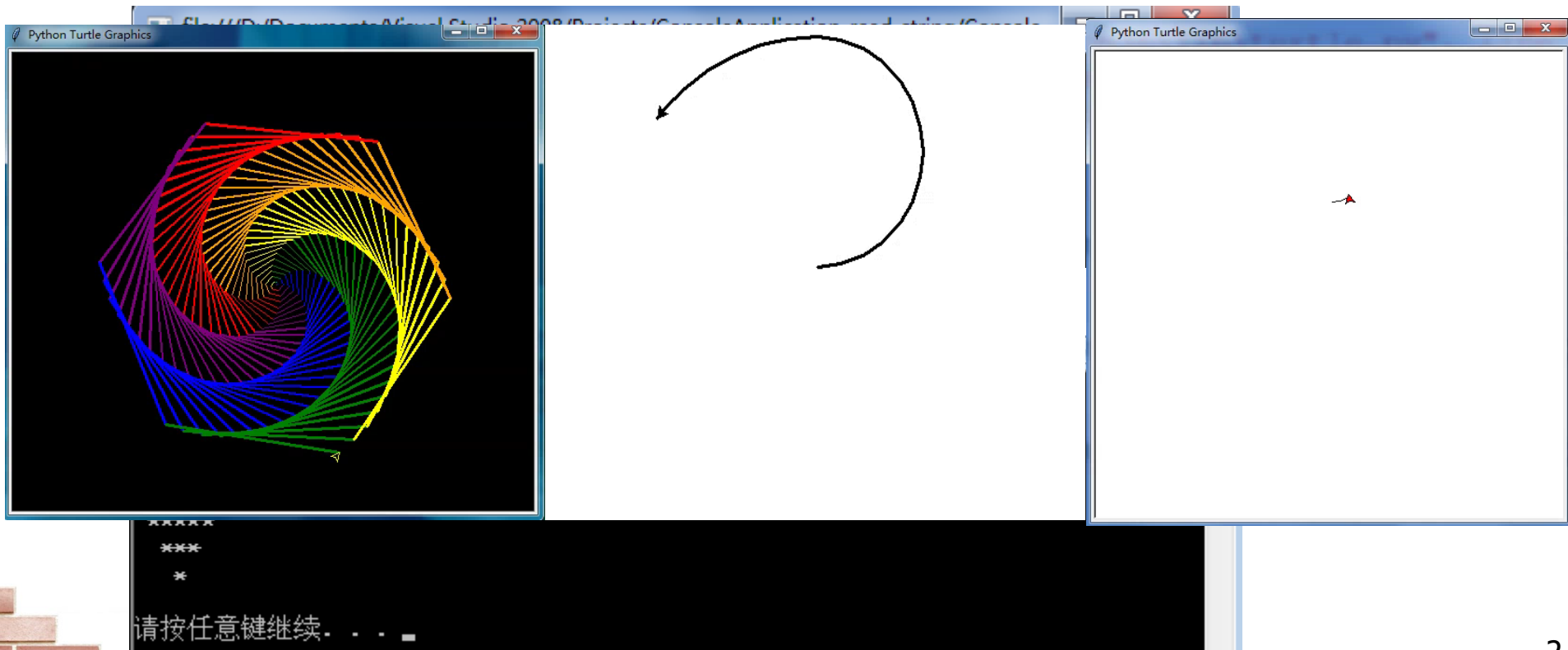
Python程序设计

人工智能学院 邓擎琮

北京师范大学

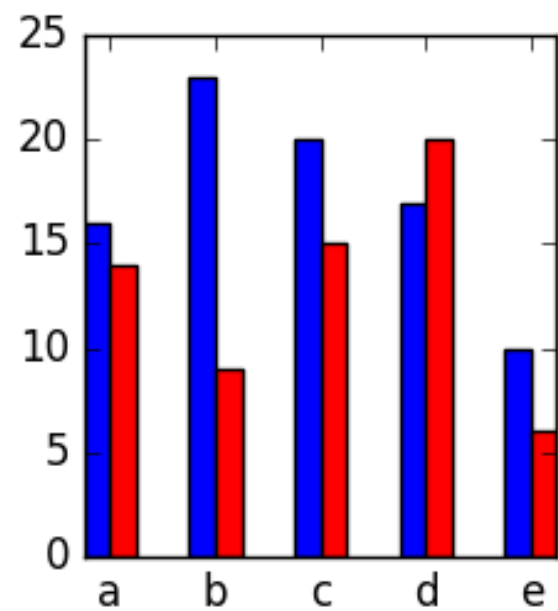
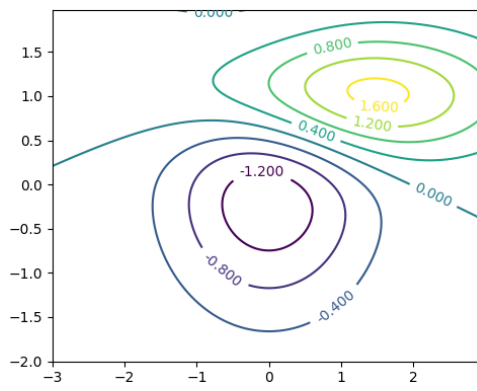
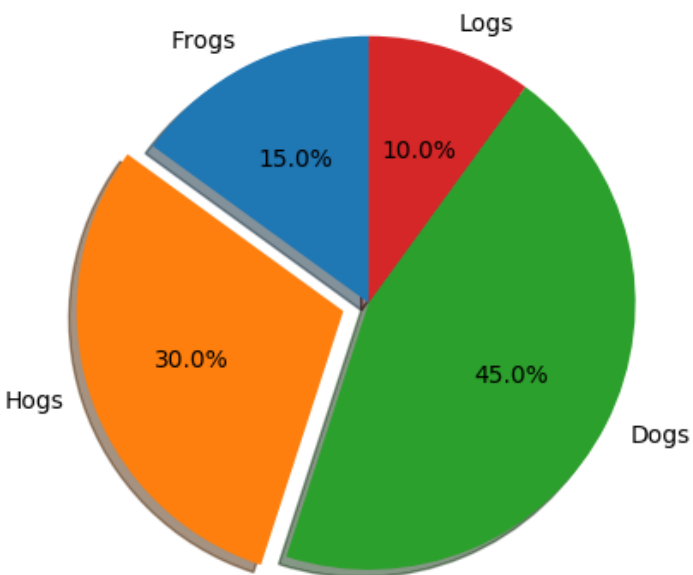
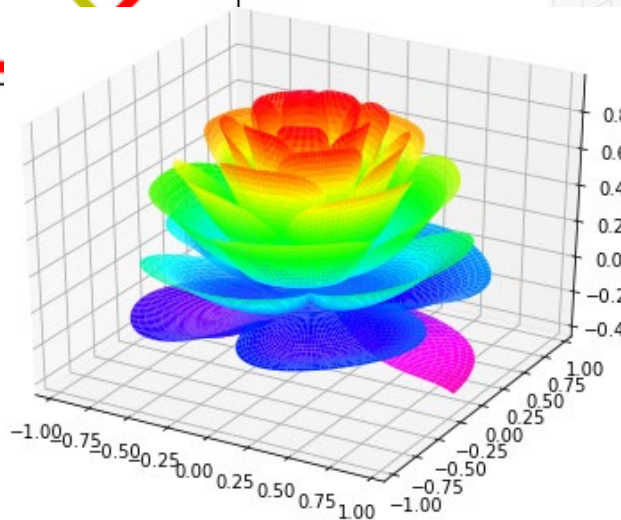
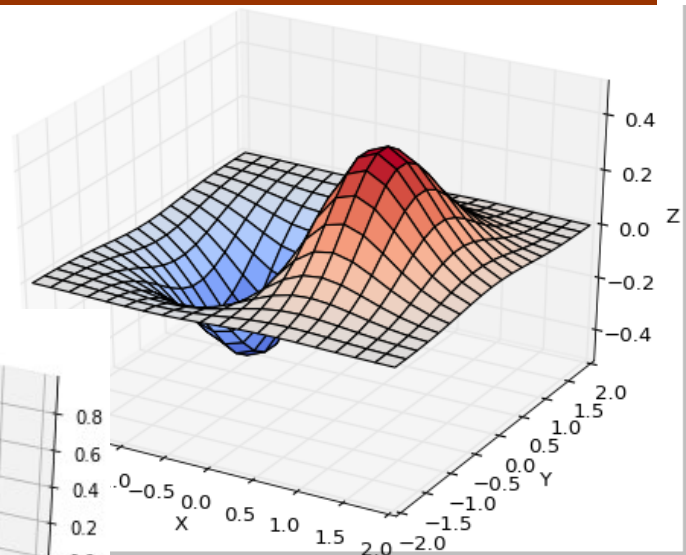
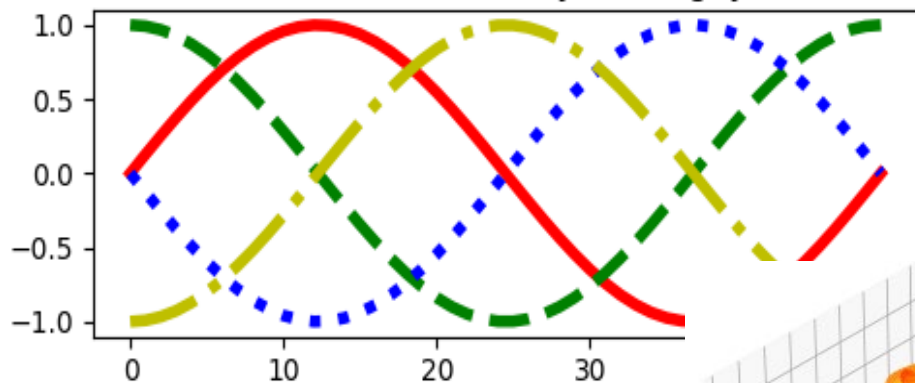
课程介绍

- 这门课上你能学到什么？
 - Python的基本语法
 - Python一些常用库（Turtle, NumPy, Matplotlib, Woldcloud, PIL, Pandas等）的使用方法



课程介绍

Set default color cycle to rgby



课程介绍

统计学院



邱东

宋旭光

王亚菲

赵楠

李昕

席玮

肖尧

朱力行

童行伟

金蛟



李慧

何元珍

郭旭

张永林

陈梦根

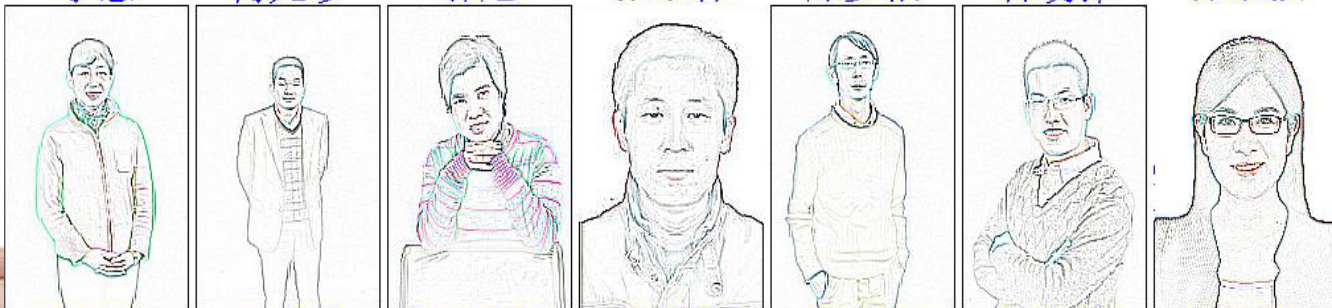
石峻骅

孙永强

徐军

张勋

李勇



张淑梅

吕光明

杜勇宏

赵俊龙

吕绍川

段小刚

牛翠珍

课程介绍



绘制公式对应的曲线

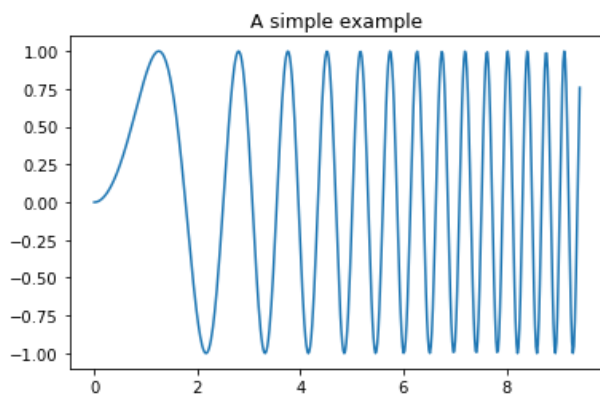
给定一个公式，画出对应的曲线图

1. 一个Latex公式

$$y = \sin(x^2)$$

2. 对应的Python程序和运行结果

```
In [22]: %matplotlib inline
import matplotlib.pyplot as plt
import numpy as np
x=np.linspace(0, 3*np.pi, 500)
plt.plot(x, np.sin(x**2))
plt.title('A simple example')
plt.show()
```



3. 还可以嵌入其他元素，如图片、视频、音频、网页等等

比如图片



教材与参考书

■ 教材：

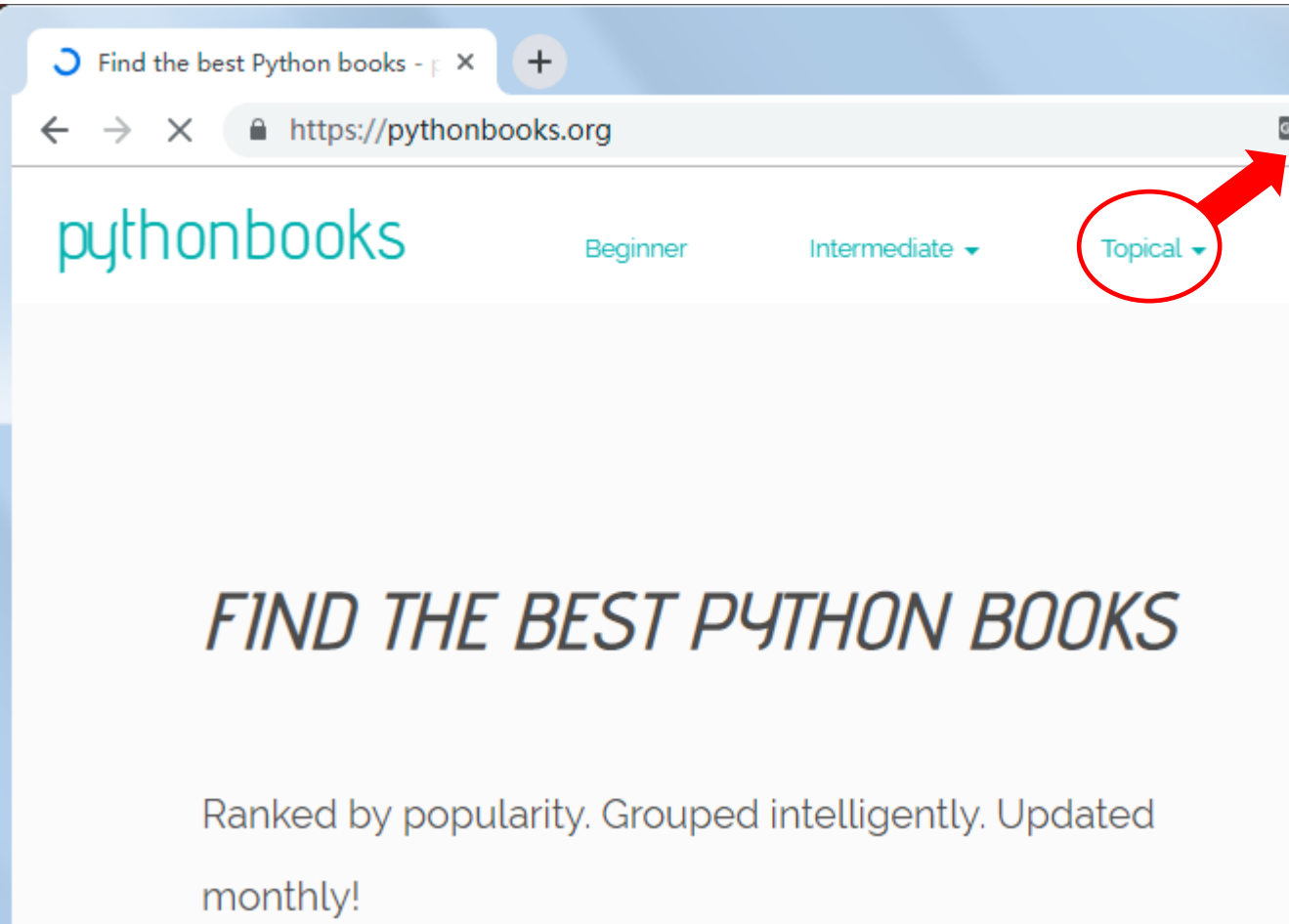
- 语言程序设计基础（第2版），嵩天、礼欣、黄天羽，高等教育出版社，2017.2
- PYTHON CRASH COURSE: A HANDS-ON, PROJECT-BASED INTRODUCTION TO PROGRAMMING, 2nd edition, 2019, by Eric Matthes

■ 参考资料：

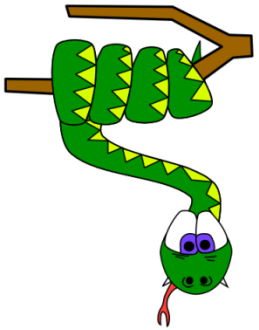
- 官方文档：<https://docs.python.org/3/>
对应的中文文档：<https://docs.python.org/zh-cn/3/>
- 推荐书单：<https://pythonbooks.org/>
- MOOC课程：北理工，嵩天，Python语言程序设计
<https://www.icourse163.org/course/BIT-268001>

教材与参考书

■ 推荐书单: <https://pythonbooks.org/>



- Algorithm and Data Structure
- Audio and Music
- Biology
- Computer Security
- DevOps and Testing
- Engineering
- Finance
- Geographic Information System
- GUI Development
- Graphics and Computer Vision
- High Performance Computing
- Machine Learning and AI
- Microcontrollers
- Network programming
- Natural Language Processing
- Numerical Programming and Data Mining
- Raspberry Pi
- Web Scraping
- Web Development
- All topical books



绪论

Why Python?

- Life is short, I using Python!
- Python的优点：
 - 简单：学习简单，使用简单
 - 功能强大：动态类型、自动内存管理、丰富的内置对象类型、丰富的内置库工具、丰富的第三方工具……
 - 应用广泛：数据库、图形图像、科学计算、web开发、操作系统等，支持多个学科，被各行各业的人员使用
 - 跨平台

TIOBE编程语言排行榜-长期走势

Programming Language	2023	2018	2013	2008	2003	1998	1993	1988
Python	1	4	8	7	13	27	17	-
C	2	2	1	2	2	1	1	1
Java	3	1	2	1	1	19	-	-
C++	4	3	4	4	3	2	2	6
C#	5	5	5	8	10	-	-	-
Visual Basic	6	15	-	-	-	-	-	-
JavaScript	7	7	10	9	8	22	-	-
Assembly language	8	12	-	-	-	-	-	-
SQL	9	251	-	-	7	-	-	-
PHP	10	8	6	5	6	-	-	-
Objective-C	17	18	3	46	48	-	-	-
Ada	28	28	16	18	15	8	8	2
Lisp	31	31	13	15	14	7	4	3
Pascal	227	136	15	20	99	12	3	14
(Visual) Basic	-	-	7	3	4	3	6	5

Who is using Python?

Google

YouTube

NYSE



YAHOO!



ubuntu

知乎

搜 狐
SOHU.com



realnetworks

豆瓣 douban

Tencent 腾讯

Fermilab

Lawrence Livermore National Laboratory

Python也被列为黑客应该学会的五种语言之一(Python,Java,C/C++,Perl和LISP)

Python的历史

■ Python起源

- 1989年末，Guido van Rossum为了打发圣诞节的无聊，开始创造一种新的解释型语言。
- 第一个Python解释器于1991年诞生
- Python的名称来自于Guido挚爱的电视剧Monty Python's Flying Circus。



■ 理念：

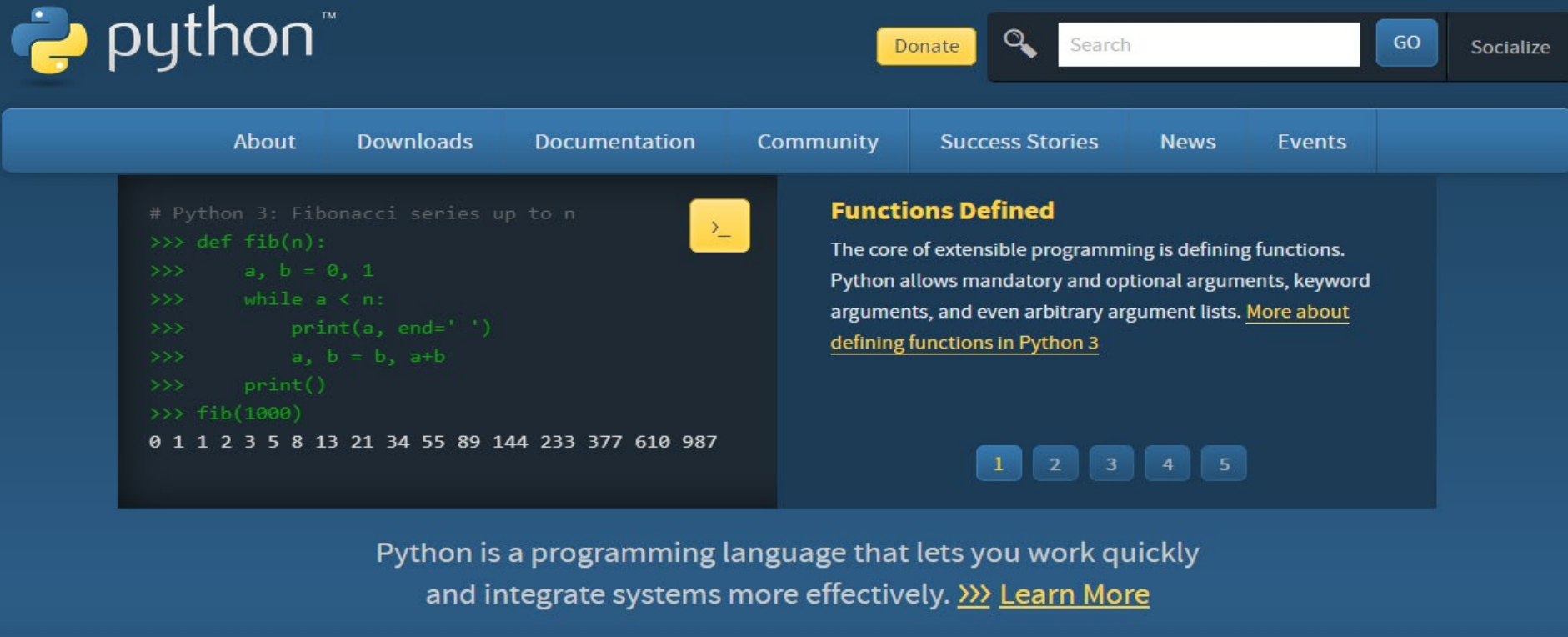
“There is only one way to do it, the right way”

-----Guido van Rossum

“简单”、“明确”、“优雅”

Python的官网

■ <https://www.python.org/>



The screenshot shows the Python.org homepage with a dark blue header and a lighter blue navigation bar. The header includes the Python logo, a 'Donate' button, a search bar, and a 'Socialize' button. The navigation bar has links for 'About', 'Downloads', 'Documentation', 'Community', 'Success Stories', 'News', and 'Events'. The main content area features a code editor with a Fibonacci sequence example, a 'Functions Defined' section, and a footer with four columns: 'Get Started', 'Download', 'Docs', and 'Jobs'.

python™

Donate

Search

GO

Socialize

About Downloads Documentation Community Success Stories News Events

```
# Python 3: Fibonacci series up to n
>>> def fib(n):
>>>     a, b = 0, 1
>>>     while a < n:
>>>         print(a, end=' ')
>>>         a, b = b, a+b
>>>     print()
>>> fib(1000)
0 1 1 2 3 5 8 13 21 34 55 89 144 233 377 610 987
```

Functions Defined

The core of extensible programming is defining functions. Python allows mandatory and optional arguments, keyword arguments, and even arbitrary argument lists. [More about defining functions in Python 3](#)

1 2 3 4 5

Python is a programming language that lets you work quickly and integrate systems more effectively. [>>> Learn More](#)

Get Started

Whether you're new to programming or an experienced developer, it's easy to learn and use Python.

[Start with our Beginner's Guide](#)

Download

Python source code and installers are available for download for all versions!

Latest: [Python 3.11.1](#)

Docs

Documentation for Python's standard library, along with tutorials and guides, are available online.

docs.python.org

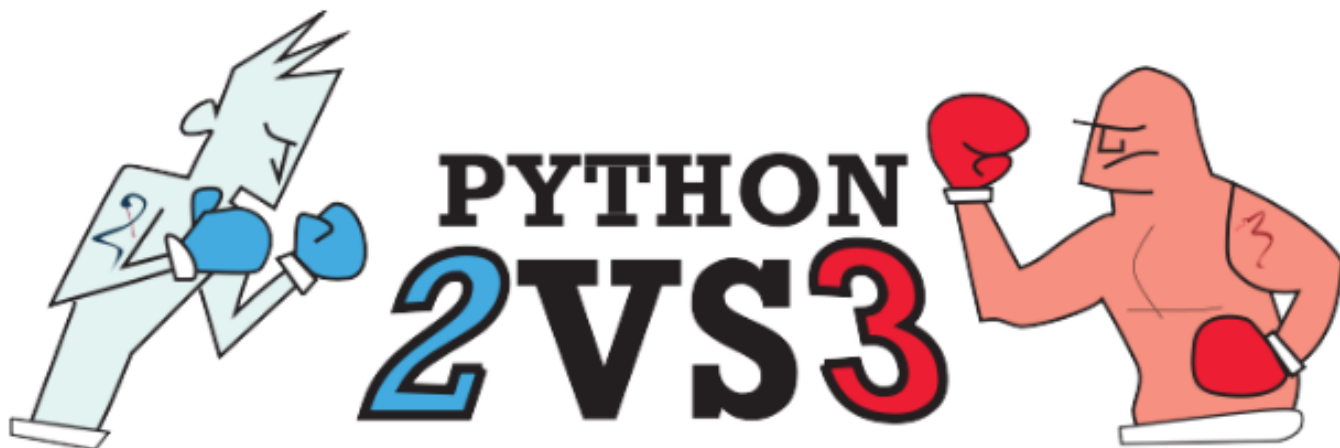
Jobs

Looking for work or have a Python related position that you're trying to hire for? Our **relaunched community-run job board** is the place to go.

jobs.python.org

Python的版本

- Python 2 （2.7是最后版本，官方已在2021年停止支持）
- Python 3 （不向下兼容）



Python的开发环境

- IDLE: Python内置IDE (随python安装包提供)
- eclipse + pydev插件 / VS Code
- PyCharm (web开发)
- Anaconda ··· ← 推荐
 - 跨平台
 - 更新较快
 - 安装第三方库简单, 且已安装丰富的第三方库 (如科学计算库)
 - 集成了conda、Spyder、Jupyter Notebook等
 - Python2和3可以共存
 -

Anaconda安装

■ Anaconda:

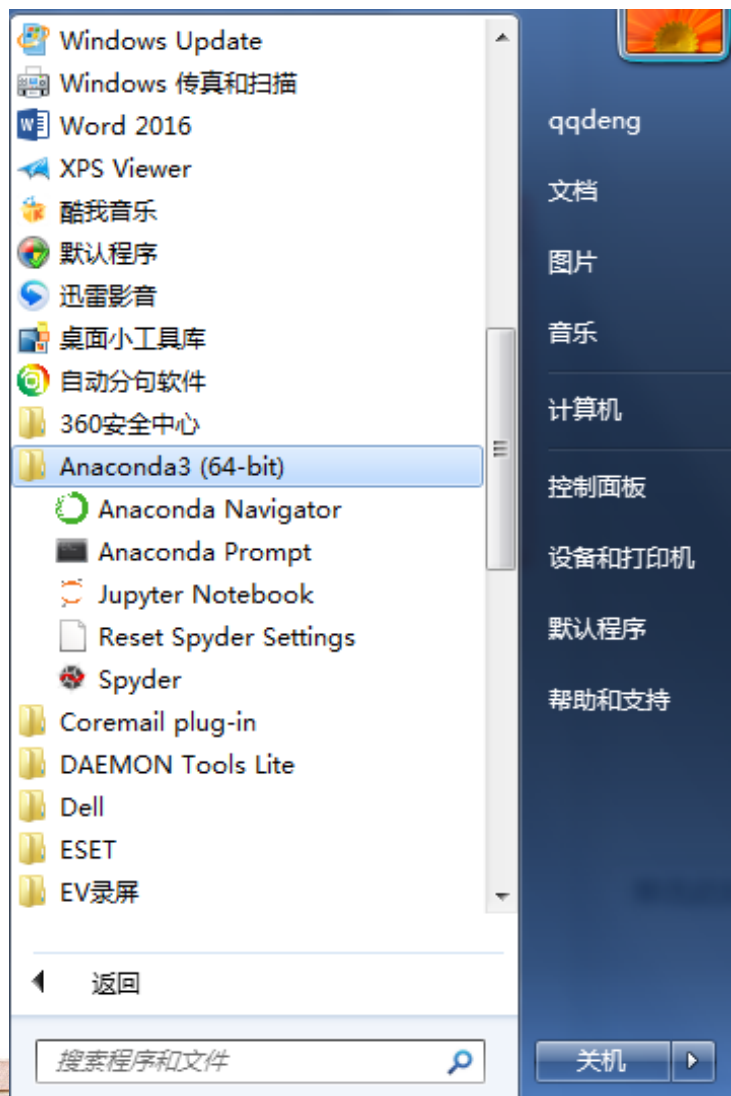
<https://www.anaconda.com/products/individual>

Windows 	MacOS 	Linux 
Python 3.9	Python 3.9	Python 3.9
64-Bit Graphical Installer (594 MB)	64-Bit Graphical Installer (591 MB)	64-Bit (x86) Installer (659 MB)
32-Bit Graphical Installer (488 MB)	64-Bit Command Line Installer (584 MB)	64-Bit (Power8 and Power9) Installer (367 MB)
	64-Bit (M1) Graphical Installer (316 MB)	64-Bit (AWS Graviton2 / ARM64) Installer (568 MB)
	64-Bit (M1) Command Line Installer (305 MB)	64-bit (Linux on IBM Z & LinuxONE) Installer (280 MB)

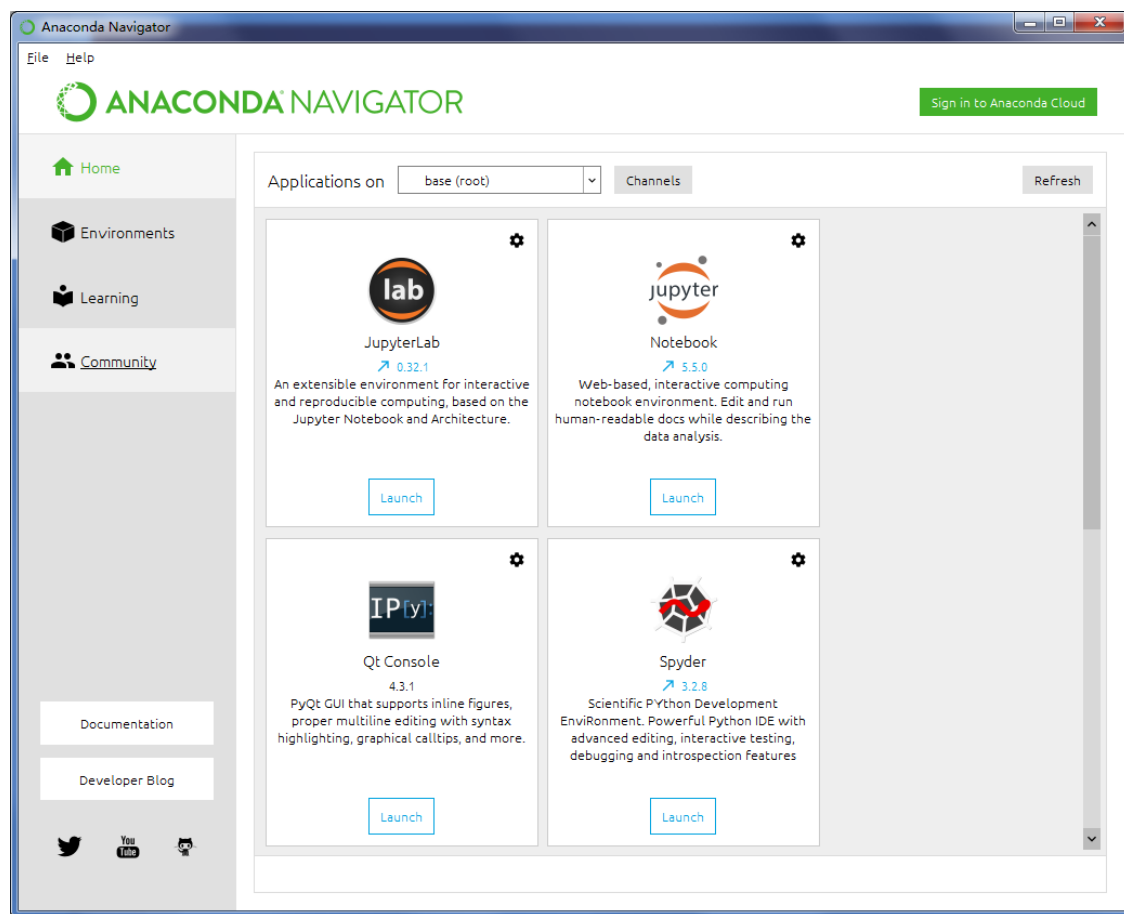
■ 历史版本下载: <https://repo.continuum.io/archive/>

使用Anaconda

Windows:



Mac:



Python语言基础

- 变量与赋值
- 数据类型简介
- 运算符和表达式
- 函数和模块简介

变量

- 在程序中创建的名字，用来表示程序中的某些“物体”

```
>>> #variable
>>> PI = 3.1415926
>>> myString = 'is a mathematic circular constant'
>>> print(PI,myString)
3.1415926 is a mathematic circular constant
```

变量名命名规则

- 标识符：允许作为变量名或其他对象名称的有效符号

- 首字符是字母或下划线
- 其余可以是字母、下划线、数字
- 名字是区分大小写的
- 名字不能是**关键字、预定义标识符**

```
>>> PI = 3.1415926
>>> pi = 'another value'
>>> print(PI)
3.1415926
>>> print(pi)
another value
```

- 关键字：Python中的特殊单词，用来表示Python解释器的命令

and	as	assert	break	class	continue	def	del
elif	else	except	exec	finally	for	from	global
if	import	in	is	lambda	not	or	pass
print	raise	return	try	while	with	yield	

变量命名规则

■ 关键字、预定义标识符

```
1 # -*- coding: utf-8 -*-
2 """
3 Created on Fri Sep  8 10:30:22 2017
4
5 @author: qqdeng
6 """
7 #绘制一朵彩色花
8
9 import turtle
10
11 t = turtle.Pen()
12 t.pensize(5)
13 colors = ['red', 'yellow', 'blue']
14
15 n=turtle.numinput('输入', '花瓣数:', 3, 3, 100)
16
17 for i in range(int(n)):
18     if i==n-1 and n%3==1:
19         t.pencolor(colors[1])
20     else:
21         t.pencolor(colors[i%3])
22     t.circle(100)
23     t.left(360/n)
24
25 turtle.done()
```

变量名命名规则2

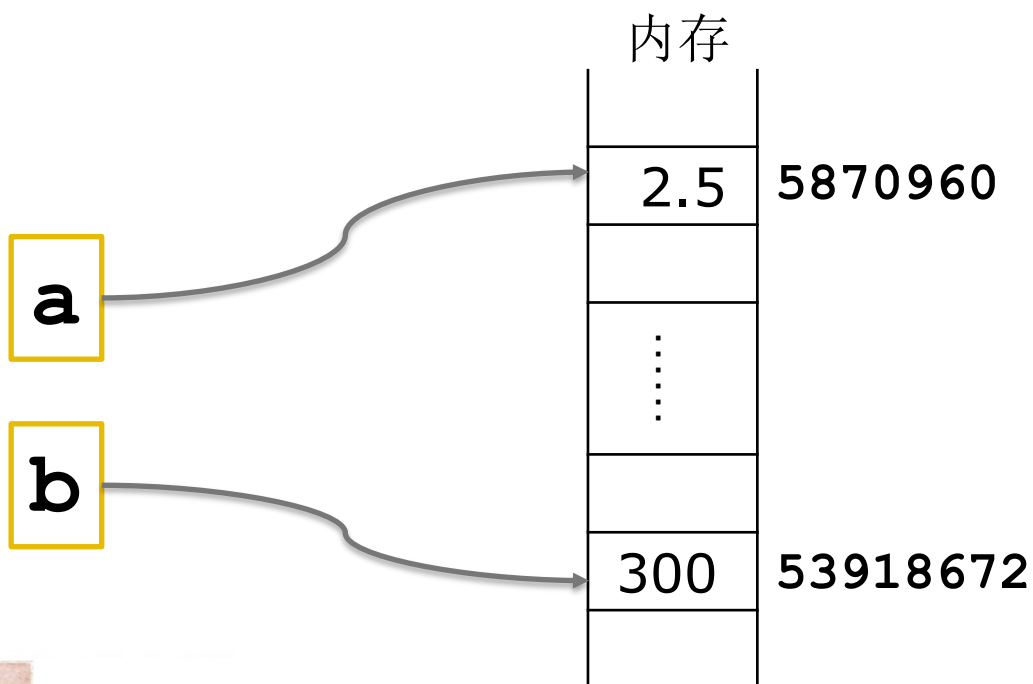
- 约定俗成的规定：
 - 大写的变量名为常量
 - 以下划线开头的为私有变量
 - 变量名要见名识义
 - 变量名中包含多个单词时可采用骆驼拼写法或下划线分隔方式：
`myString/my_string`

赋值

- 变量赋值时同时获得类型和值
 - 不需要显示声明，根据值确定类型
 - 通过“引用”把变量和值（对象）连接起来

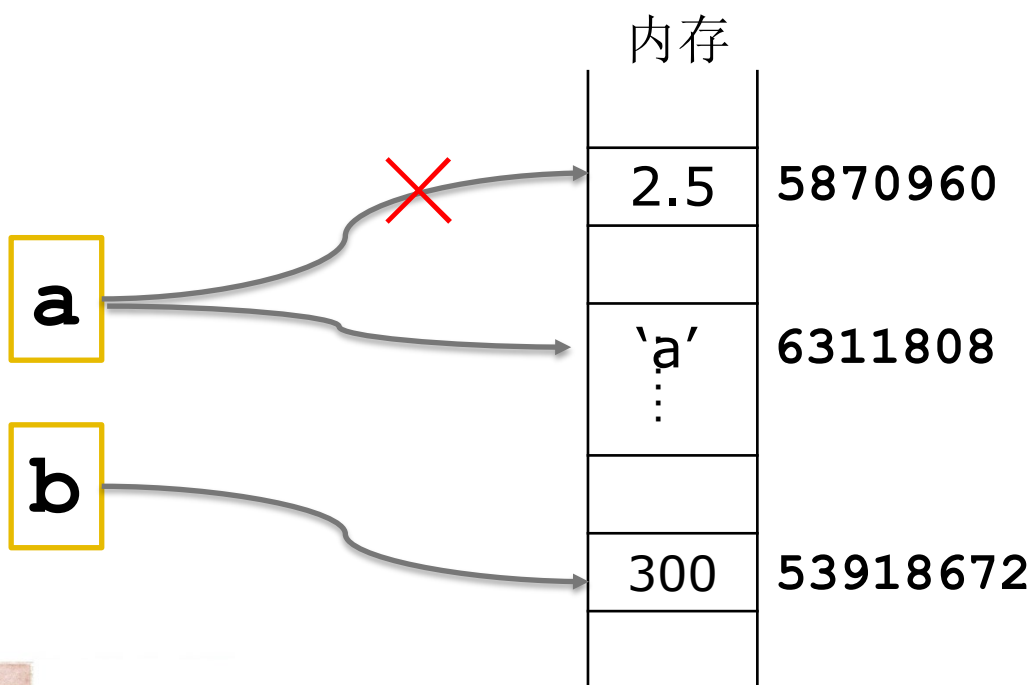
```
>>> a=2.5
>>> b=300
>>> type(a)
<class 'float'>
>>> type(b)
<class 'int'>
```

```
>>> id(a)
5870960
>>> id(b)
53918672
```



赋值2

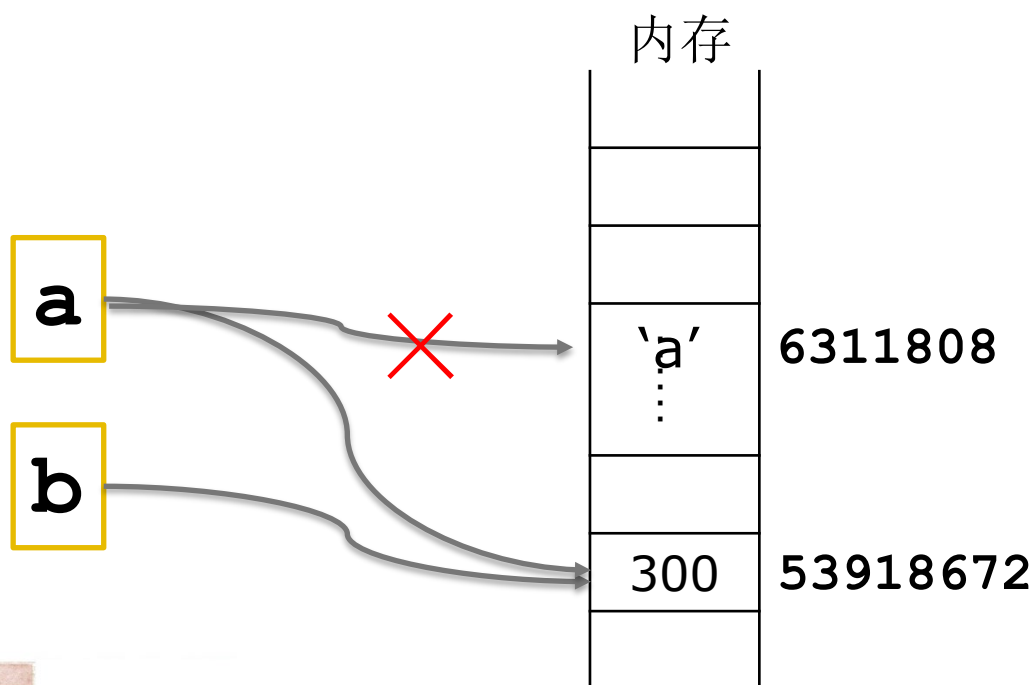
- 通过“=”给变量重新赋值，不改变原始对象的值，而是创建一个新对象，并指向它。



```
>>> a='a'
>>> id(a)
6311808
>>> type(a)
<class 'str'>
```

赋值3

- 一个变量可以=另外一个变量，那么它们指向的那个对象的引用就增加了一个



```
>>> a=b
>>> type(a)
<class 'int'>
>>> id(a)
53918672
```

更多赋值方式

■ 多重赋值

$a = b = 3.5$

■ 多元赋值

$a, b, c = 1, 2, 3$

$t = [1, 2, 3]$

$a, b, c = t$

$t = (1, 2, 3)$

$a, b, c = t$

多元赋值可用于交换

$x = 1$

$y = 2$

$x, y = y, x$

■ 自变赋值（增量赋值）

■ $+=$ $-=$ $*=$ $/=$ $\%=$ $**=$ $<<=$ $>>=$ $\&=$ $\^=$ $|=$

$a /= 5$ 即 $a = a/5$

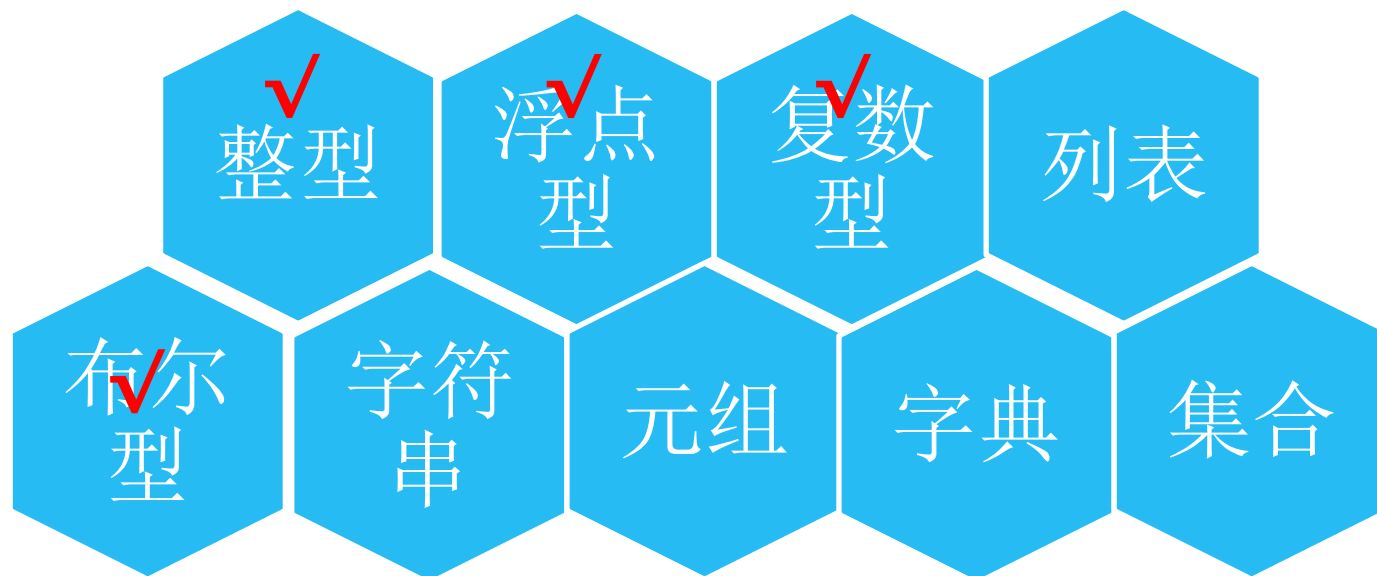
$a **= 2$ 即 $a = a**2$

Python语言基础

- 变量与赋值 ✓
- 数据类型简介
- 运算符和表达式
- 函数和模块简介

Python数据类型

数值类型



容器类型

整型

- 支持的长度与内存大小有关，可以处理很大很大的整数，基本无溢出问题。
- 十进制、八进制（前缀0o/0O）、十六进制（前缀0x/0X）、二进制（前缀0b/0B）

```
>>> type(12)
<class 'int'>
>>> 0o10
8
>>> 0O10
8
>>> type(0o10)
<class 'int'>
```

```
>>> 0x10
16
>>> type(0x10)
<class 'int'>
>>> 0b10
2
>>> type(0b10)
<class 'int'>
```


布尔型

- 整型的子类：只有两个值： True、False
- 本质上分别是整型的1和0

```
>>> x=True
>>> int(x)
1
>>> type(x)
<class 'bool'>
>>> y=False
>>> int(y)
0
```

浮点型

- 即数学中的实数
- 可以用科学计数法表示
- 有精度限制

```
>>> 5.8
5.8
>>> 3.475e3
3475.0
>>> -9.8e-2
-0.098
>>> type(-9.8e-2)
<class 'float'>
```

```
>>> 2.2222222222222222222222222222
2.22222222222222222222222222223
```

```
>>> 5.5555555555555555555555555555
5.5555555555555555555555555555
```

```
>>> 6.6666666666666666666666666666
6.6666666666666666666666666667
```

```
>>> 1.1+2.2==3.3
False
```

复数型

- 实部+虚部j（大写或小写）

```
>>> 2.4+5.6j
(2.4+5.6j)
>>> type(2.4+5.6j)
<class 'complex'>
>>> 3J
```

- 取复数的实/虚部

- 复数.real
- 复数.imag

```
3j
>>> type(3J)
<class 'complex'>
>>> 5+0j
(5+0j)
```

- 复数的共轭

- 复数.conjugate()

```
>>> type(5+0j)
<class 'complex'>
>>> x=2.4+5.6j
>>> x.imag
5.6
>>> x.real
2.4
>>> x.conjugate()
(2.4-5.6j)
```

类型转换

- int、float、complex数据可以混合运算
- 在不同类型数据做计算时, Python自动转换:

int → float → complex

```
>>> type(2+31+4.5+(1+2j))  
<class 'complex'>  
>>> 2+31+4.5+(1+2j)  
(38.5+2j)
```

类型转换2

- 手动类型转换:利用int(), float(), complex()

```
>>> int('1'), int(2.3)
(1, 2)
```

```
>>> float('3.33'), float(5)
(3.33, 5.0)
```

```
>>> complex(4), complex('6.6')
((4+0j), (6.6+0j))
```


容器类型

有顺序的容器
(序列类型)

字符串

- 元素均为字符，不可变类型，单引号/双引号/三引号为界

元组

- 元素可以不同类型，不可变类型，()为界

列表

- 元素可以不同类型，可变类型，[]为界

集合

- 元素可以不同类型，但不重复，有可变和不可变两种集合

字典

- 每个元素为键值对 (**key:value**) 组成，映射类型，可变类型，{}为界

无顺序的容器

字符串 str

■ 外形:

- 单引号'
- 双引号"
- 三引号'''/''''''

■ 内容: 字符

■ 特点: 序列, 不可变

```
>>> type('Hello world!')
```

```
<class 'str'>
```

```
>>> type("Hello world!")
```

```
<class 'str'>
```

```
>>> type("""Hello world!""")
```

```
<class 'str'>
```

```
>>> type("""Hello world!""")
```

```
<class 'str'>
```

```
>>> print("""Hello)
world!""")
```

```
Hello)
```

```
world!
```

三引号可使字符串跨行

元组 tuple

- 基本形式：(元素1, 元素2, ..., 元素n) -----元素可以不同类型
- 特点：序列，不可变

```
>>> a = (1, 'a', 3.5, True, (1, 2), 1)
>>> type(a)
<class 'tuple'>
>>> a
(1, 'a', 3.5, True, (1, 2), 1)
```

列表 list

- 基本形式：[元素1, 元素2, ..., 元素n] -----元素可以不同类型
- 特点：序列，可变

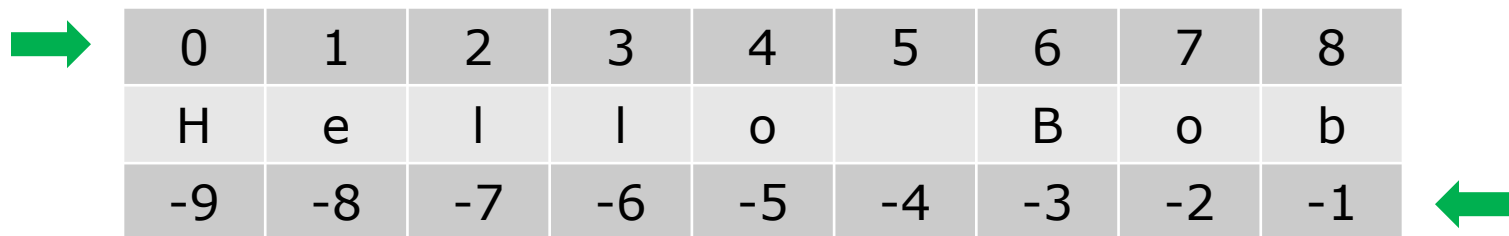
```
>>> b=[1,2,1,'apple']  
>>> type(b)  
<class 'list'>  
>>> b  
[1, 2, 1, 'apple']
```

序列基本操作：索引访问操作

■ 序列可通过**位置索引（下标）**访问每个元素

- 从前往后的索引：**0**代表第一个位置。
- 从后往前的索引：**-1**代表倒数第一个位置。

■ 例如：若 `s1 = 'Hello Bob'`，则索引可以：



0	1	2	3	4	5	6	7	8
H	e	l	l	o		B	o	b
-9	-8	-7	-6	-5	-4	-3	-2	-1

`s1[0]`或`s1[-9]`是'H'

`s1[5]`或`s1[-4]`是' '

`s1[9]`或`s1[-10]`越界出错

序列索引访问操作更多示例

```
>>> a = (1, 'a', 3.5, True, (1, 2), 1)
```

```
>>> a[4]
```

```
(1, 2)
```

```
>>> a[4][1]
```

```
2
```

```
>>> b=[1,2,1,'apple']
```

```
>>> b[-1]
```

```
'apple'
```

集合 set/ frozenset

■ 集合：

- 类似数学中的集合概念：不重复元素数据集
- 元素可以不同类型
- 无序（不是序列类型）
- 两种集合：可变集合set，不可变集合frozenset

集合 set/ frozenset

■ 集合：

- 类似数学中的集合概念：不重复元素数据集
- 元素可以不同类型
- 无序（不是序列类型）
- 两种集合：可变集合set，不可变集合frozenset

```
>>> a={' a', 1, ' b', 1, -1}
>>> a
{1, ' b', ' a', -1}
```

```
>>> b=set([' a', 1, 1])
>>> b
{1, ' a'}
```

```
>>> c=frozenset(' ab3ab' )
>>> c
frozenset({' b', ' a', ' 3' })
```

```
>>> a[0]
TypeError: 'set' object is not subscriptable
```

字典 dict

- 基本形式: {key1:value1, key2:value2,.....}

- 特点: key不重复, 通过key查找value, 可变, 无序

- 示例: 快递单号与物流信息、通讯录, 学生信息管理等

```
>>> a={2016001:['张三',98,100], 2016002:['李四',75,80]}
```

```
>>> a  
{2016001: ['张三', 98, 100], 2016002: ['李四', 75, 80]}
```

```
>>> a[2016001]  
['张三', 98, 100]
```

可变 VS 不可变

- 可变：可增删改内容 （列表、set、字典）
- 不可变：不可增删改内容 （字符串、元组、frozenset）



可变示例

```
>>> b=[1,2,1,'apple']  
>>> b[-1]='banana'  
>>> b  
[1, 2, 1, 'banana']
```

```
>>> b.remove(1)  
>>> b  
[2, 1, 'banana']
```

```
>>> b.append('a')  
>>> b  
[2, 1, 'banana', 'a']
```

不可变示例

```
>>> name='Tom'
```

```
>>> name[1]='i'
```

```
Traceback (most recent call last):
```

```
  File "<pyshell#11>", line 1, in <module>
```

```
    name[1]='i'
```

```
TypeError: 'str' object does not support item assignment
```

Python语言基础

- 变量与赋值 ✓
- 数据类型简介 ✓
- 运算符和表达式
- 函数和模块简介

表达式

- 用运算符连接各种类型数据的式子就是表达式

算术运算符

乘方	**
正负号	+ -
乘除	* /
整除	//
取余	%
加减	+ -

位运算符

取反	~
与	&
或	
异或	^
左移	<<
右移	>>

比较运算符

小于	<
大于	>
小于等于	<=
大于等于	>=
等于	=
不等于	!=
is	
is not	

逻辑运算符

非	not
与	and
或	or

运算符的优先级顺序

运算符	描述
**	幂(指数)
~ + -	补，一元加号和减号(方法名的最后两个+@和-@)
* / % //	乘，除，取模和地板除
+ -	加法和减法
>> <<	左，右转向按位
&	位元“与”
^	按位异或“和常规”或
<= < > >=	比较运算符
<> == !=	操作符比较
= %= /= //= -= += *= **=	赋值运算符
is is not	操作符标识
in not in	操作符成员
not or and	逻辑运算符

详见表2-1

比较运算

■ 连写形式

```
>>> 3<4<2 #same as (3<4) and (4<2)
False
>>> (3<4) and (4<2)
False
>>> 4>3==3 #same as (4>3) and (3==3)
True
>>> (4>3) and (3==3)
True
>>> 1<3<5!=2<7
True
```

Python语言基础

- 变量与赋值 ✓
- 数据类型简介 ✓
- 运算符和表达式 ✓
- 函数和模块简介

函数和模块简介

- 内置函数（直接使用）
- 标准模块（导入后使用）
- 扩展模块（安装、导入后使用）

Python内置函数

abs()	dict()	help()	min()	setattr()
all()	dir()	hex()	next()	slice()
any()	divmod()	id() ✓	object()	sorted()
ascii()	enumerate()	input() ✓	oct()	staticmethod()
bin()	eval()	int() ✓	open()	str()
bool()	exec()	isinstance()	ord()	sum()
bytearray()	filter()	issubclass()	pow()	super()
bytes()	float() ✓	iter()	print() ✓	tuple()
callable()	format()	len()	property()	type() ✓
chr()	frozenset() ✓	list()	range()	vars()
classmethod()	getattr()	locals()	repr()	zip()
compile()	globals()	map()	reversed()	__import__()
complex() ✓	hasattr()	max()	round()	
delattr()	hash()	memoryview()	set() ✓	

dir(__builtins__)查看内置函数清单，help(一个函数名)查看该函数文档

dir(__builtins__)与help函数

```
In [1]: dir(__builtins__)
```

```
Out[1]:
```

```
['ArithmeticError',  
 'AssertionError',  
 .....
```

```
['abs',  
 'all',  
 'any',  
 .....,  
 'type',  
 'vars',  
 'zip']
```

```
In [2]: help(len)
```

```
Help on built-in function len in module builtins:
```

```
len(obj, /)
```

```
Return the number of items in a container.
```

```
In [3]: len?
```

```
Signature: len(obj, /)
```

```
Docstring: Return the number of items in a  
container.
```

```
Type: builtin_function_or_method
```

Python标准模块

■ 官方文档: <https://docs.python.org/3/library/>

- 9. Numeric and Mathematical Modules
 - 9.1. `numbers` — Numeric abstract base classes
 - 9.2. `math` — Mathematical functions
 - 9.3. `cmath` — Mathematical functions for complex numbers
 - 9.4. `decimal` — Decimal fixed point and floating point arithmetic
 - 9.5. `fractions` — Rational numbers
 - 9.6. `random` — Generate pseudo-random numbers
 - 9.7. `statistics` — Mathematical statistics functions
- 10. Functional Programming Modules
 - 10.1. `itertools` — Functions creating iterators for efficient looping
 - 10.2. `functools` — Higher-order functions and operations on callable objects
 - 10.3. `operator` — Standard operators as functions
- 11. File and Directory Access
 - 11.1. `pathlib` — Object-oriented filesystem paths
 - 11.2. `os.path` — Common pathname manipulations
 - 11.3. `fileinput` — Iterate over lines from multiple input streams
 - 11.4. `stat` — Interpreting `stat()` results
 - 11.5. `filecmp` — File and Directory Comparisons
 - 11.6. `tempfile` — Generate temporary files and directories
 - 11.7. `glob` — Unix style pathname pattern expansion
 - 11.8. `fnmatch` — Unix filename pattern matching
 - 11.9. `linecache` — Random access to text lines
 - 11.10. `shutil` — High-level file operations
 - 11.11. `macpath` — Mac OS 9 path manipulation functions

Python标准模块

■ 导入和使用模块的基本形式:

<code>import 模块名</code>	<code>#导入模块</code>
<code>模块名.函数名</code>	<code>#使用模块中的函数</code>
<code>模块名.变量名</code>	<code>#使用模块中的变量</code>

```
>>> import math
>>> math.sin(math.pi/2.0)
1.0
```

■ 也可以直接导入变量和函数:

`from 模块名 import 变量名/函数名`

```
>>> from math import sin
>>> sin(3.14159/2)
0.999999999999991198
```

math模块应用举例

- 求解一元二次方程 $x^2 + 5x + 6 = 0$ 的解

```
import math
a=1;b=5;c=6
x1=(-b+math.sqrt(b*b-4*a*c))/(2*a)
x2=(-b-math.sqrt(b*b-4*a*c))/(2*a)
print('方程x**x+5*x+6的解为: ',x1, x2)
```

方程x**x+5*x+6的解为: -2.0 -3.0

常用数学模块

- math模块：数学函数
- cmath模块：复数运算函数
- decimal模块：高精度数值运算
- fractions模块：分数运算
- random模块：随机数

参见4.3、4.4、4.7-4.9

dir, help函数

dir,help示例

```
>>> import random
>>> dir(random)
['BPF', 'LOG4', 'NV_MAGICCONST', 'RECIP_BPF', 'Random', 'SG_MAGICCONST', 'System
Random', 'TWOPI', '_BuiltinMethodType', '_MethodType', '_Sequence', '_Set', '_a
ll__', '__builtins__', '__cached__', '__doc__', '__file__', '__loader__', '__nam
e__', '__package__', '__spec__', '_acos', '_bisect', '_ceil', '_cos', '_e', '_ex
p', '_inst', '_itertools', '_log', '_pi', '_random', '_sha512', '_sin', '_sqrt',
'_test', '_test_generator', '_urandom', '_warn', 'betavariate', 'choice', 'choi
ces', 'expovariate', 'gammavariate', 'gauss', 'getrandbits', 'getstate', 'lognor
mvariate', 'normalvariate', 'paretovariate', 'randint', 'random', 'randrange', '
sample', 'seed', 'setstate', 'shuffle', 'triangular', 'uniform', 'vonmisesvari
ate', 'weibullvariate']
>>> help(random.randint)
Help on method randint in module random:
```

```
randint(a, b) method of random.Random instance
    Return random integer in range [a, b], including both end points.
```

```
>>> random.randint(2,5)
```

5

```
>>> random.randint(3,5)
```

4

扩展模块

- <https://pypi.org/>

Find, install and publish Python packages
with the Python Package Index

Search projects



Or [browse projects](#)

396,796 projects

3,733,943 releases

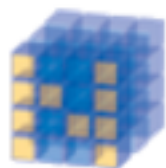
6,620,744 files

618,374 users

- <https://github.com/vinta/awesome-python> （不全！）
- 对应的中文网站： <https://github.com/jobbole/awesome-python-cn> （不全！）

扩展模块

■ 本课程涉及模块:



NumPy

Base N-dimensional
array package



BeautifulSoup



Matplotlib

Comprehensive 2D
Plotting



pandas

Data structures &
analysis

PIL



SciPy library

Fundamental
library for scientific
computing

wordcloud

jieba

练习

- 对绘制彩色花的程序进行修改，得到不同效果的花朵，例如：

